

SD5 — Distributed Lock Manager (lease + fencing)

Oracle: aggregator-listed LLD/HLD; the *concepts* (lease, fencing) are probed inside every other design too — this file is leverage. JD tie-in (`../0.req/oracle_senior_ic3.pdf`): "distributed systems ... scalability, resiliency" — locks are where interviewers test whether you know what breaks. Tags: `[hot path]` · `[async]` · `[control plane]` · **?** = follow-up + answer.

Index

0. Time budget
 1. Why this is hard
 2. Crux
 3. Clarifying questions
 4. FRs / NFRs
 5. Back-of-envelope
 6. High-level design
 7. API
 8. Data model
 9. Deep dives (3)
 10. Hygiene + ops
 11. Pop-up questions
 12. Trade-offs + defeaters
 13. Wrap-up + card
 14. References
-

§0 Time budget (often a 25–30 min segment, not a full round)

Clarify 3 · why-hard 3 · design 8 · client code 5 · deep dive 8 · wrap 3.

§1 Why this is hard

naive: `SETNX lock:job1 → do work → DEL lock:job1`

breaks at: holder crashes → lock held forever; add TTL → GC pause outlives TTL → two holders

- Naive design: Redis `SET key val NX` — no TTL → crashed holder deadlocks everyone
- Add TTL → new failure: holder pauses (GC, VM freeze, packet loss) past TTL; lock expires; second client acquires; first wakes **still believing it holds the lock** → two writers
- Minimum primitive that fixes it: **fencing token** — a number that increases on every acquisition, checked *at the protected resource*, not at the lock service
- What the user wants: "at most one active worker per resource, and progress when workers die"

- Cost of being wrong: double-processing (payments, cert issuance) or stuck pipelines

🔑 "A distributed lock without fencing is a *hint*, not a guarantee. The resource must be able to reject a stale holder — that's the crux (Kleppmann's argument, and I agree with it)."

\$2 Crux

- Safety: at most one holder **whose writes the resource accepts** at any time
- Liveness: crashed holder's lock frees itself (lease TTL)
- The lock service alone can only give liveness; safety needs the fencing check downstream
- Critical points: monotonic token generation · TTL $\gg$ max pause (but small enough for recovery SLA) · heartbeat renewal · resource-side check

\$3 Clarifying questions

Question	Assumed
Correctness or efficiency lock?	correctness (double-run is harmful, not just wasteful)
Scale?	10k locks, 1k acquire/s, hold p50 5 s
Reentrant? blocking or try?	tryAcquire + waiters via watch; non-reentrant v1
Fairness?	best-effort FIFO, not strict
Multi-region?	no — a lock spanning regions is a design smell

\$4 FRs / NFRs

- FR: `acquire(resource, ttl) → lease{token}, renew(lease), release(lease), watch(resource)`
- NFR: acquire p99 < 20 ms · availability 99.95% (lock service down = work pauses, must not corrupt) · safety absolute (never two accepted holders) · lock state survives service-node failure

\$5 Back-of-envelope

- 1k acquire/s $\times$ ~100 B state = trivial data; this is a **latency + correctness** service, not a throughput one
- Consensus backend (3–5 nodes) handles ~10k writes/s (etcd order of magnitude) — one cluster is enough; shard by resource prefix if ever needed

\$6 High-level design

```
[hot path] Client - acquire(resource, ttl) → Lock API (stateless) → consensus store (etcd/ZK-style, Raft, 3-5 nodes)
                                     | key: /locks/{resource} = {holder, token,
lease_until}
[async]   client heartbeat: renew every ttl/3 —┐ token = store's global revision (monotonic for
free)
[hot path] Client - write(data, token) → Resource (DB/storage) - accepts only token ≥ last seen
[control plane] watchers notified on release/expiry → next waiter tries (no thundering herd: notify one)
```

- Acquire = conditional create: succeed iff key absent or lease expired; token = Raft log index / etcd `mod_revision` → monotonic without extra machinery
- Expiry enforced by the store's lease mechanism (etcd leases / ZK ephemeral + session), not client clocks

\$7 API

```
POST /v1/locks/{resource}:acquire {ttl_s: 30, wait_ms: 0|N}
→ 200 {lease_id, fencing_token, expires_at} | 409 {holder, expires_at}
POST /v1/leases/{lease_id}:renew → 200 {expires_at} | 410 (lost – STOP WORKING)
POST /v1/leases/{lease_id}:release → 204
GET /v1/locks/{resource}:watch → long-poll/stream: released events
```

- Contract line to say: "a 410 on renew means you may no longer act; abort or re-acquire — and your writes were fenced anyway."

\$8 Data model

```
-- if backed by an RDBMS instead of etcd (small scale / already-have-Postgres answer)
CREATE TABLE locks (
  resource      text PRIMARY KEY,
  holder        text,
  fencing       bigint NOT NULL DEFAULT 0,  -- monotonic per resource
  lease_until   timestampz
);
-- acquire (atomic):
-- UPDATE locks SET holder=$who, fencing=fencing+1, lease_until=now()+$ttl
-- WHERE resource=$r AND (holder IS NULL OR lease_until < now())
-- RETURNING fencing;
```

\$9 Deep dives

9.1 Client: acquire / heartbeat / fenced write

```

import threading
import time

class Lease:
    def __init__(self, client, resource, lease_id, token, ttl_s):
        self.client, self.resource = client, resource
        self.lease_id, self.token, self.ttl = lease_id, token, ttl_s
        self.lost = threading.Event()
        threading.Thread(target=self._heartbeat, daemon=True).start()

    def _heartbeat(self):
        while not self.lost.is_set():
            time.sleep(self.ttl / 3)
            try:
                self.client.renew(self.lease_id)
            except LeaseLostError:
                # 410
                self.lost.set()
                # holder must check this flag

    def guard(self):
        """Call before/around every side effect."""
        if self.lost.is_set():
            raise LeaseLostError(self.resource)
        return self.token
        # pass to the resource with the write

def do_protected_work(lock_client, db):
    lease = lock_client.acquire("jobs/rotate-ca", ttl_s=30) # raises if held
    try:
        token = lease.guard()
        db.update_if_fresh(token, ...) # resource-side fencing check
    finally:
        lock_client.release(lease.lease_id)

# The guard is advisory; the db-side token check is the actual safety.

```

```

public final class Lease implements AutoCloseable {
    private final LockClient client;
    private final String leaseId;
    private final long token;                // fencing token
    private final long ttlMs;
    private volatile boolean lost = false;
    private final ScheduledExecutorService hb = Executors.newSingleThreadScheduledExecutor();

    Lease(LockClient c, String leaseId, long token, long ttlMs) {
        this.client = c; this.leaseId = leaseId; this.token = token; this.ttlMs = ttlMs;
        hb.scheduleAtFixedRate(() -> {
            try { client.renew(leaseId); }
            catch (LeaseLostException e) { lost = true; hb.shutdown(); }
        }, ttlMs / 3, ttlMs / 3, TimeUnit.MILLISECONDS);
    }

    public long guard() {
        if (lost) throw new LeaseLostException("stop working");
        return token;
    }

    @Override public void close() { hb.shutdownNow(); client.release(leaseId); }
}

```

Resource-side check (the part that makes it safe):

```

UPDATE resource_state SET data=$1, last_token=$2
WHERE resource=$3 AND last_token < $2;    -- 0 rows updated => stale holder, reject

```

9.2 Why Redlock is not the answer (say this before they ask)

- Redlock = SET NX PX on 5 independent Redis nodes, majority wins
- Failure: it depends on **bounded clock drift and bounded pauses** — exactly the assumptions that fail (GC pause between check and write). No fencing token; a paused holder still writes
- Verdict to state: "Redis lock = fine for *efficiency* (avoid duplicate cache fill); for *correctness* use a consensus store **and** fencing at the resource. Kleppmann vs antirez — side with fencing."

9.3 Herd, fairness, and waiters

- Naive watch: all waiters wake on release → thundering herd on the store
- ZK pattern: each waiter creates a sequential ephemeral node, watches **only its predecessor** → one wakeup, natural FIFO
- etcd pattern: waiters watch the key at a revision; server notifies in order
- Cap waiters per resource (e.g., 100) → 429 beyond; a deep queue means the design should probably be a work queue instead of a lock

§10 Hygiene + ops

- TTL sizing: $> \max(\text{GC pause, network blip})$ but $< \text{recovery SLA}$; 15–30 s typical; heartbeat $\text{ttl}/3$
- Metrics: acquire latency, contention (waiters per resource), lease_expired count, **fenced_write_rejected** (should be ~ 0 ; nonzero = pauses happening — investigate)
- Lock service down: clients keep working until their lease expires, then stop (fail-safe); new work pauses — alarm, don't improvise
- Test: SIGSTOP a holder past TTL, resume — assert its write is rejected and exactly one holder's writes landed

§11 Pop-up questions

- *Reentrant?* → "Track (holder, count) server-side; renew bumps count only conceptually — keep v1 non-reentrant, most 'reentrant' needs are a code smell across services."
- *Read locks / RW?* → "Shared lease set with writer exclusivity; complexity doubles; only if a real use case."
- *Lock across two resources?* → "Acquire in global lexicographic order to dodge deadlock; better: redesign to one lock."
- *Why not DB advisory locks?* → "pg_advisory_lock is fine intra-Postgres; ties lock lifetime to a session/connection — pool recycling silently releases it. Explicit lease table or etcd is more honest."
- *Clock skew?* → "Expiry evaluated by the store's clock only; clients never compare their clock to lease_until — they react to renew failures."

§12 Trade-offs + defeaters

Decision	Chosen	Alternative	Flips when
Backend	consensus store (etcd/ZK)	Postgres row lease	you already run PG and scale is low
Safety	fencing at resource	trust the lock	never
Waiters	watch predecessor	poll	tiny scale

- Defeaters: Redlock-as-correctness · TTL-only without fencing · "the GIL/one region makes it safe"

§13 Wrap-up + card

- Pitch: lease in a consensus store, monotonic fencing token, heartbeat renewal, token checked at the resource
- 5 numbers: TTL 30 s · heartbeat 10 s · acquire p99 20 ms · 3–5 store nodes · waiter cap 100
- 3 failures: GC pause past TTL (fence) · holder crash (TTL frees) · herd on release (watch predecessor)
- Done when: can write `Lease` + the fenced UPDATE from memory; can argue Redlock in 60 s

§14 References

- Kleppmann, "How to do distributed locking"; antirez's reply (know both sides)
- etcd lock/lease docs; ZooKeeper recipes (sequential ephemeral nodes)
- Chubby paper (Google) — locks as a coarse-grained service